

MySQL应用教程

第十章 INNODB的事务隔离级别

INNODB的基本封锁

对于普通的select语句，INNODB不会加任何锁，事务可以通过以下语句显式给记录集加共享锁或排它锁：

- 共享锁（S）：`select * from table_name where ... lock in share mode;`
- 排他锁（X）：`select * from table_name where ... for update。`

对于写操作语句，INNODB会加排它锁。

INNODB的封锁粒度

- ▶ INNODB支持表锁和行锁

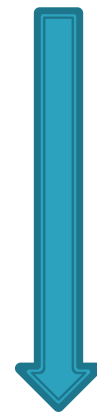
当检索时使用了索引，就加行锁；

当检索时没有用到索引，就加表锁。

INNODB的事务隔离级别

- ▶ 未提交度 (READ-UNCOMMITTED)
- ▶ 提交读(READ-COMMITTED)
- ▶ 可重复读(REPEATABLE-READ)
- ▶ 序列化(SERIALIZABLE)

隔离
级别



低

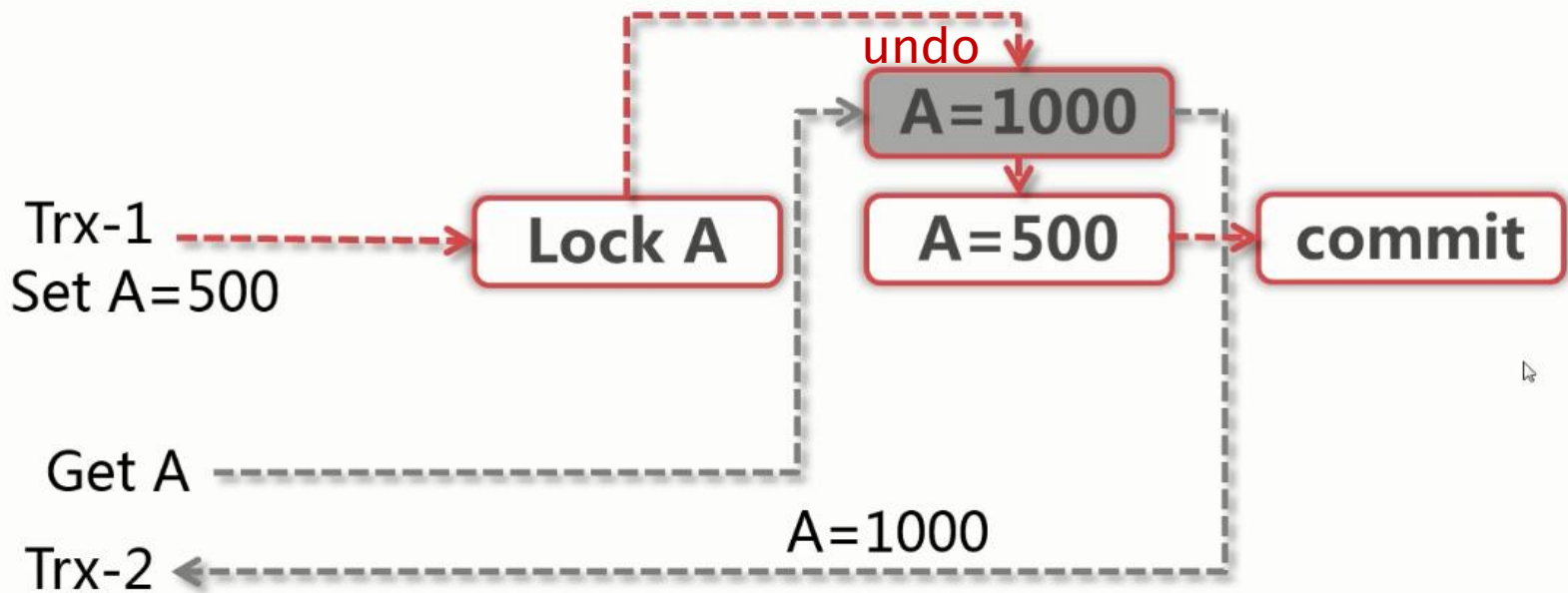
高

可重复读和提交读隔离级别的实现方式

MVCC（多版本并发控制）

- MVCC是通过undo日志来实现
- 读不加锁, 做“快照读”

MVCC(多版本并发控制)



1 未提交读（READ-UNCOMMITTED）

- ▶ 提供了事务间最小限度的隔离。

通过名字我们就可以知道，在这种事务隔离级别下，一个事务可以读到另外一个事务未提交的数据。

- ▶ 实现原理

读不加锁，写加排它锁（X锁）



直接读磁盘中的数据

事务一	事务二
BEGIN; R(A)=25	
	W(A)=20;
R(A)=20	
	ROLLBACK;
COMMIT;	

事务一共查询了两次，在两次查询的过程中，事务二对数据进行了修改，并未提交（commit）。但是事务一的第二次查询查到了事务二的修改结果。

2 提交读（READ-COMMITTED）

- 提交读的隔离级别比可重复读差。

通过名字也可以分析出，在一个事务修改数据过程中，如果事务还没提交，其他事务不能读该数据

- 实现原理

读不加锁, 做“快照读”

并总是读最新的快照

写加排它锁(X锁)



事务一	事务二
BEGIN; R(A)=25	
	BEGIN; W(A)=20;
R(A)=25	
	COMMIT;
R(A)=20	
COMMIT;	

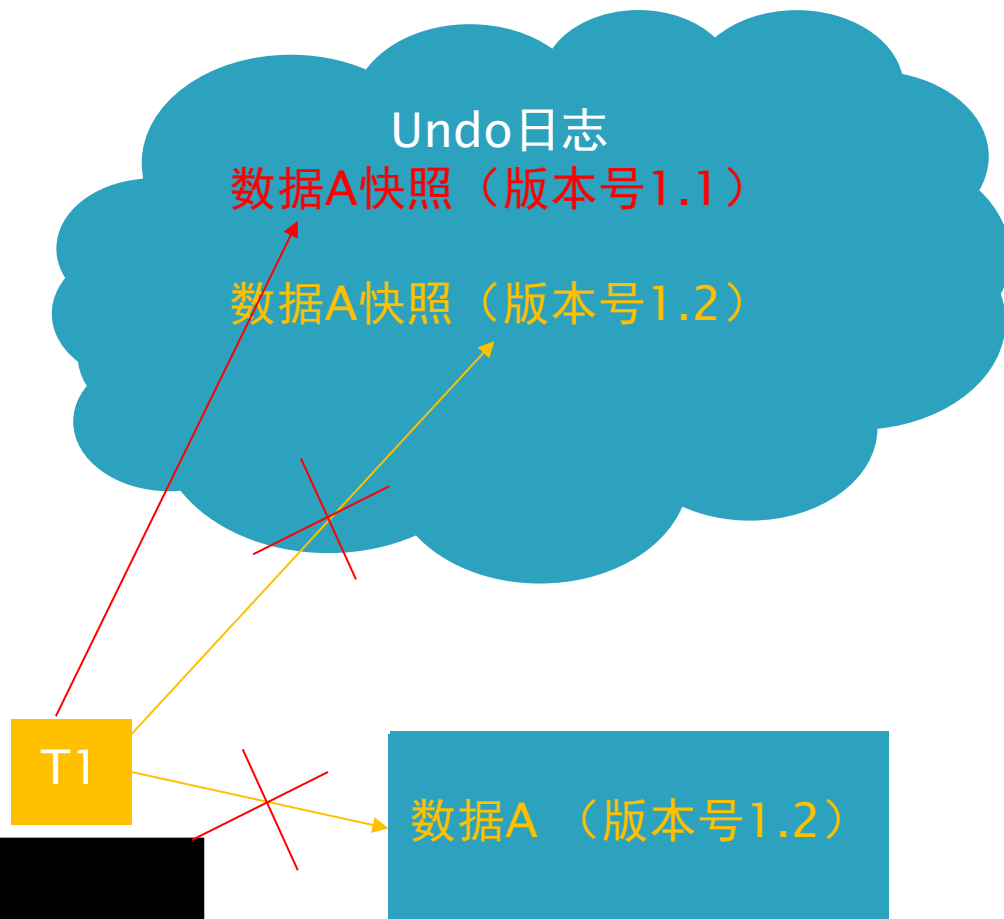
提交读这种隔离级别保证了读到的任何数据都是提交的数据，避免
)，但是不保证事务重新读的时候能读到相同的

3 可重复度 (REPEATABLE-READ)

- ▶ 保证同一事务在执行同条SELECT语句数次，结果总是相同的。
- ▶ 实现原理
 - 读不加锁, 做“快照读”
 - 并总是读事务开始第一次读的那个版本的快照
 - 写加排它锁(X锁)

INNODB的MVCC多版本并发控制

读不加锁, 做“快照读”
并总是读事务开始第一次读的那个版本的
快照。



事务一	事务二
BEGIN; R(A)=25	
	BEGIN; W(A)=20;
R(A)=25	
	COMMIT;
R(A)=25	
COMMIT;	

在上面的例子中，只有在事务一提交之后，事务二才能更改该行数据。所以，只要在事务一从开始到结束的这段时间内，无论他读取该行数据多少次，结果都是一样的。

事务一	事务二
BEGIN; SELECT * FROM users WHERE id=9; (结果为NULL)	
	INSERT users(id,username)VALUES(9, 'kiki');
	COMMIT;
INSERT users(id,username)VALUES(9, 'linmy'); (报错冲突)	
COMMIT;	

事务1的内心是：怎么回事？查了id=9为空集，insert id=9告诉我PK冲突

这是已提交事务2对事务1产生的影响，这个影响叫做“**幻读**”。

4 序列化 (SERIALIZABLE)

- ▶ 用户之间通过一个接一个顺序执行当前的事务，提供了事务之间最大限度的隔离。
- ▶ 实现原理
 - 读加共享锁 (S锁)
 - 写加排它锁 (X锁)

事务一	事务二
BEGIN; SELECT * FROM users WHERE id=7;	
	INSERT users(id,username)VALUES(7, 'kiki');
	COMMIT;
INSERT users(id,username)VALUES(7,' linmy');	
COMMIT;	

INNODB隔离级别的设置和查看

1.查看当前会话隔离级别

```
select @@ transaction_isolation;
```

2.查看系统当前隔离级别

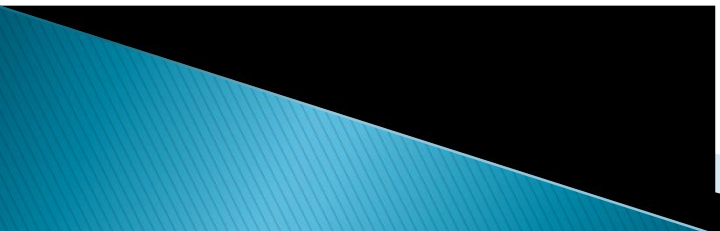
```
select @@global. transaction_isolation;
```

3.设置当前会话隔离级别

```
set session transaction isolation level  
repeatable read;
```

4.设置系统当前隔离级别

```
set global transaction isolation level  
repeatable read;
```



实验：

对上述隔离级别分别进行实验，并用加锁原理来解释各个例子中的结果。

